

OpenYAM + dimOS

A practical first run with YOLOE, GraspGenX and agentic control

Reviewed 19 September 2026 | Revision 2 | Tscharn Labs

Verdict: Keep this hardware and this model choice. The architecture is sensible, but the original guide is not a validated, copy-and-run setup. It contains configuration errors and overstates GPU readiness and collision protection. This revision fixes the source-level issues and makes the remaining physical measurements explicit.

Locked hardware	Role in the first run
Ubuntu + RTX 2070 Max-Q	Local vision, grasp inference, planning and motor control. Verify actual VRAM and driver; performance on this laptop is not yet measured.
OpenYAM + gripper	One bolted-down arm. Use the existing dimOS Damiao adapter.
D435i on the bench	The only RGB-D input to detection and grasp generation. Rigid, fixed mount looking into the workspace.
60 fps wrist camera	Optional operator view initially. Its advertised frame rate does not establish depth capability or dimOS integration.

Smallest useful outcome

Detect one opaque, rigid object; generate grasps locally; perform one supervised pick and place into a measured clear area; then trigger the same skills through the agent. No training, ROS2, Docker, second computer or wrist-camera calibration is required for this baseline.

Local robotics, cloud language model

YOLOE and GraspGenX run on the Ubuntu machine. The standard dimOS MCP agent is a local client of a hosted language model and needs internet plus an API key. Fully offline language-model inference is a separate integration and is not validated here.

What was actually validated

The 24-page original was reviewed against the dimOS source at commit **da5f4d50c33f2eabcd1ee95a15434b0954bcd92b** and its pinned GraspGenX dependency. The Python blueprint below was syntax-checked and its named APIs/configuration fields were checked in source. No installation, GPU inference, calibration or physical motion was executed on your equipment. Treat the hardware checkpoints as acceptance tests, not completed results.

Reading order: corrections (2), install (3), cameras (4), measurements (5), configuration and blueprint (6-8), no-motion test (9), first pick (10), agent (11), troubleshooting and sources (12).

Corrections that change the setup

Original issue	Correction in this revision
YOLOE default mode omitted	Explicitly set <code>YoloePromptMode.PROMPT</code> . The scene module otherwise defaults to LRPC and loads prompt-free weights; <code>scan_objects</code> passes text prompts, which that checkpoint cannot accept.
Wrong gripper joint placeholder	Use <code>OPENYAM_GRIPPER_JOINT</code> . Its actual value is <code>arm/gripper</code> , not <code>gripper</code> .
Zero-size gripper geometry	Positive extents and fingertip depth are required by <code>GraspGenXConfig</code> . Zero placeholders fail validation. Record real geometry and the TCP transform.
Claimed comfortable 3.1-4.6 GB peak	Remove this unsupported estimate. Measure a prompted scan and real grasp inference together on the 2070 Max-Q. CUDA support alone does not validate the full model.
Base-pose source patch	Unnecessary: <code>make_openyam_model_config().model_copy(update={...})</code> can set the existing <code>base_pose</code> field without changing <code>upstream config.py</code> .
Missing RealSense build	The current camera module is a Rust executable with its own Nix/librealsense build. Python installation and <code>rs-enumerate-devices</code> alone are insufficient.
Collision-free pick implied	The final approach, retreat and place legs use <code>move_linear(check_collision=False)</code> . A detected object list is not a complete bench collision map.
YOLOE described as not open-vocabulary	YOLOE supports open-vocabulary text prompts. For <code>detector_backend=yoloe</code> , <code>segmentation_backend=yolo</code> reuses its masks; other detectors use a YOLOE box segmenter.
Only two measurements claimed	Also establish bench collision geometry and gripper feedback thresholds. A motor stall is not proof of a stable grasp.
Registration test assumed to pass	Locally it rewrites <code>all_blueprints.py</code> and can intentionally fail because the generated file is uncommitted. Review that specific diff; other errors remain failures.
Camera-only check moved the arm	Removed <code>go_init</code> from perception validation. The new proposals blueprint contains no motor controller.

Also removed: assumptions about your exact CPU/RAM/Ubuntu version, claims that teleop was already completed, automatic handover coordinates, and the suggestion that unplugging CAN is an emergency stop. USB/CAN disconnection is not a dependable torque-off mechanism.

1. Install and check the GPU

Use Ubuntu 22.04 or 24.04 x86_64 for the reviewed installation path. Confirm your installed version rather than assuming it. Keep the arm powered off during software preparation. [1]

```
lsb_release -a
uname -m
nvidia-smi
free -h
df -h .
```

The 2070 Max-Q is Turing (sm_75), normally 8 GB. The reviewed lock uses PyTorch 2.7.1 / CUDA 12.8 in the isolated GraspGenX runtime. Use an NVIDIA driver compatible with that runtime; R570 or newer is the straightforward CUDA 12.8 baseline. A full CUDA toolkit is not needed just to use the PyTorch wheels. [4, 8]

Pinned developer checkout

Run from the directory where you want the project. The official installer supplies system dependencies. Existing checkouts are reused without pulling, so pin before installing. Do not overwrite another project checkout.

```
sudo apt-get update
sudo apt-get install -y git git-lfs curl
GIT_LFS_SKIP_SMUDGE=1 git clone https://github.com/dimensionalOS/dimos.git
cd dimos
git checkout da5f4d50c33f2eabcd1ee95a15434b0954bcd92b
git switch -c openyam-first-run
bash scripts/install.sh --mode dev --no-nix \
  --extras manipulation --project-dir "$PWD"
source .venv/bin/activate
uv --version
dimos --help
```

Use uv 0.9.25 or newer. Keep the installer-selected environment; if adding the manipulation extra later, use **uv sync --locked --extra manipulation --group tests**. The manipulation extra already includes vision and agents. Its CPU extra does not mean CPU-only Torch wheels on Linux x86_64. Avoid casually upgrading Torch or Ultralytics. [1, 4]

Verify actual CUDA execution

```
python - <<'CHECK'
import torch
print(torch.__version__, torch.version.cuda)
assert torch.cuda.is_available(), "CUDA unavailable"
print(torch.cuda.get_device_name(0))
print(torch.cuda.get_device_capability(0))
x = torch.randn(128, 128, device="cuda")
print((x @ x).isfinite().all().item())
CHECK
```

This checks the host environment only. GraspGenX uses a separate locked environment, so its real inference test on page 9 is still required. The pinned upstream model includes a pure-PyTorch attention path; no confirmed architectural blocker was found, but no 2070 benchmark is claimed.

2. Bring up the fixed D435i

Bolt the camera mount to the same bench as the arm. Aim for a clear oblique view of the grasp area with useful depth on the object and tabletop. Use USB 3, avoid hubs initially, and keep the wrist camera disconnected until this path is stable.

The extra build the original guide missed

Keep the Ubuntu Python environment from page 3. Use Nix only for the RealSense native build below. Install Nix via [the official Linux installer](#) if missing; enable nix-command and flakes in your Nix configuration. Open a fresh shell afterward, return to the dimos checkout, and activate .venv. [3]

```
nix --version
# From the dimos repository root; parentheses preserve your directory.
(
  cd dimos/hardware/sensors/camera/realsense/rust
  nix --extra-experimental-features "nix-command flakes" \
    develop path:. -c cargo build --release
)
(
  cd dimos/hardware/sensors/camera/realsense/rust
  nix --extra-experimental-features "nix-command flakes" \
    develop path:. -c rs-enumerate-devices
)
dimos run real-sense-camera-vis
```

Record the D435i serial number. If access is denied, install the official librealsense udev rules for your Ubuntu release and reconnect the camera; running the robot stack as root is not the fix. See the [official Linux installation instructions](#).

First-run camera settings

Setting	Baseline
Color/depth	848 x 480 at 15 fps, aligned depth to color. Confirm the device supports this profile.
IMU	Off. It is not required for a camera rigidly mounted to the bench.
Point cloud stream	Off in the minimal blueprint. Scene registration projects aligned RGB-D into object clouds itself.
Wrist camera	Independent viewing only. Do not mix wrist RGB with D435i depth; they have different viewpoints and calibration.

The shipped viewer is useful for color/depth validation, but its current blueprint does not explicitly enable the camera point-cloud stream. A missing 3D cloud there is not by itself a failed depth camera. Stop the viewer before launching another camera blueprint; only one process should own the D435i. [3]

Pass: stable aligned images, sensible depth on the chosen test object, correct serial, no USB reset loop. A 60 fps wrist camera does not imply a 60 Hz perception, grasping or agent loop.

3. Measure the rig once

Camera pose: world to camera_link

Define **world** to coincide with the arm model base: X forward, Y left, Z up. Measure the actual base datum relative to the tabletop; the table is not automatically Z=0. The camera needs a full translation and rotation, in metres and quaternion order x,y,z,w.

A tape measure gives an initial estimate. Validate against several known 3D points across the working area and at more than one height. For more accuracy, use a calibrated checkerboard/ChArUco target with a measured pose relative to the base and solvePnP, then invert/compose the result. This is a calibration workflow, not an included dimOS one-click command.

Camera optical axes differ from robot axes. solvePnP estimates target-to-optical-camera pose, not world-to-camera_link. Combine the measured target pose with the inverse solvePnP transform, then account for the RealSense camera_link-to-optical transform. Do not paste an optical-frame pose into camera_link or use a guessed pitch-only quaternion.

Acceptance: measured residuals must be smaller than the available jaw/object and tabletop clearance, with margin. For a centimetre-scale object, aim for a few millimetres, not merely a visually plausible overlay. Recalibrate after moving the camera or arm.

Gripper geometry and TCP

GraspGenX conditions its predictions on open and half-open sweep volumes. These are full box extents and centre offsets in metres in the grasp frame, not half-extents and not a substitute for full collision geometry. Determine them from the actual YAM gripper geometry/URDF and physical measurements. The pinned GraspGenX README still lists YAM support as upcoming. [4, 9]

Use approach +Z and jaw-closing X for the GraspGenX frame. Measure the rigid transform to the planning tip **gripper_tip**. dimOS computes $T_{world_tcp} = T_{world_grasp} @ T_{grasp_tcp}$. Identity is valid only if the frames truly coincide. Do not copy xArm dimensions. Extents and fingertip depth must be strictly positive.

Feedback and bench geometry

The normalized gripper API is 0=closed, 1=open even though the raw OpenYAM motor opens in the decreasing direction. Use the stock hardware checks on page 10 first to record empty-close, full-open, and held-object readbacks. Set empty_epsilon between empty-close and the smallest test object reading; check opening thresholds too. Do not disable verification just to make a failed pick report success. [6]

Record the bench collision box centre and full XYZ size, plus any pedestal, clamps or camera mount in the arm workspace. Record a clear placement TCP position. A container point-cloud centroid is not a safe placement pose: container clearance, object size and approach geometry also matter.

4. Create the measured configuration

Create **openyam_bench.json** in the dimos repository root. The nulls are deliberate missing-measurement markers: replace them before loading the custom blueprints. No example robot coordinates are supplied that could accidentally be executed.

```
{
  "camera_serial": null,
  "camera_translation_m": null,
  "camera_quaternion_xyzw": null,
  "gripper": {
    "extents_open": null,
    "offset_open": null,
    "extents_half_open": null,
    "offset_half_open": null,
    "fingertip_depth": null,
    "family": "parallel_2f"
  },
  "grasp_frame_to_tcp": null,
  "bench_center_m": null,
  "bench_size_m": null,
  "empty_epsilon": null,
  "place_tcp_m": null
}
```

Field	Expected value
camera_serial	String containing the D435i serial.
camera_translation_m	Three numbers: x, y, z of camera_link in world.
camera_quaternion_xyzw	Four numbers forming a unit quaternion.
gripper extents / offsets	Three numbers each. Full extents > 0; offsets may be zero. fingertip_depth is one positive number.
grasp_frame_to_tcp	4 x 4 matrix as four lists of four numbers. Proper rigid rotation; last row [0,0,0,1].
bench_center_m / bench_size_m	Three numbers each. Box centre (not tabletop height) and full dimensions.
empty_epsilon	Measured normalized threshold, not a distance in metres.
place_tcp_m	Three measured coordinates for the TCP at release, allowing for the held object.

The following two pages are one Python file, split only for readability. Save both code blocks, in order, to **dimos/robot/manipulators/openyam/blueprints/bench.py**. This is a proposed source-reviewed integration, not an upstream OpenYAM grasp blueprint. The existing OpenYAM teleop and planner blueprints remain the foundation.

First-run settings use text-prompted YOLOE-11L masks, on-demand scans, one detection to promote an object, no artificial 6 cm obstacle-width cap, low motion speed scales, and one grasp attempt per pick. The full stack adds no voxel mapper initially; model the actual bench explicitly on page 10.

5. Blueprint / imports and perception

Copy this block, then append the block on page 8. The proposals blueprint will use a fixed TF publisher without any motor controller. Configuration is loaded from an explicit absolute path.

```
from __future__ import annotations
import json
import os
from pathlib import Path
from dimos.agents.mcp.mcp_client import McpClient, McpClientConfig
from dimos.agents.mcp.mcp_server import McpServer
from dimos.control.coordinator import TaskConfig
from dimos.core.coordination.blueprints import autoconnect
from dimos.hardware.sensors.camera.realsense.camera import RealSenseCamera
from dimos.manipulation.grasping.grasp_gen_x.module import GraspGenXModule
from dimos.manipulation.manipulation_module import ManipulationModule
from dimos.manipulation.manipulation_skills import ManipulationSkills
from dimos.manipulation.pick_and_place_module import PickAndPlaceModule
from dimos.msgs.geometry_msgs.PoseStamped import PoseStamped
from dimos.msgs.geometry_msgs.Quaternion import Quaternion
from dimos.msgs.geometry_msgs.Transform import Transform
from dimos.msgs.geometry_msgs.Vector3 import Vector3
from dimos.perception.experimental.object_scene_registration import ObjectSceneRegistrationModule
from dimos.perception.detection.detectors.yoloe import YoloePromptMode
from dimos.protocol.tf.static_tf_publisher import StaticTfPublisher
from dimos.robot.manipulators.common.blueprints import coordinator, trajectory_task
from dimos.robot.manipulators.openyam.config import make_openyam_model_config, openyam_hardware
from dimos.robot.manipulators.openyam.joints import OPENYAM_GRIPPER_JOINT

C = json.loads(Path(os.environ["OPENYAM_BENCH_CONFIG"]).read_text())
def require_values(value):
    if value is None:
        raise ValueError("Complete all measurements in openyam_bench.json")
    if isinstance(value, dict):
        for item in value.values():
            require_values(item)
    elif isinstance(value, list):
        for item in value:
            require_values(item)
require_values(C)

_mount = Transform(
    translation=Vector3(*C["camera_translation_m"]),
    rotation=Quaternion(*C["camera_quaternion_xyzw"]),
    frame_id="world", child_frame_id="camera_link",
)

class OpenYamBenchMount(StaticTfPublisher):
    def transforms(self):
        return [_mount]

_sensing = (
    RealSenseCamera.blueprint(serial_number=C["camera_serial"],
        width=848, height=480, fps=15, align_depth_to_color=True,
        enable_pointcloud=False, enable_imu=False),
    ObjectSceneRegistrationModule.blueprint(target_frame="world",
        detector_backend="yoloe", segmentation_backend="yolo",
        prompt_mode=YoloePromptMode.PROMPT, detect_on_request=True,
        min_detections_for_permanent=1, distance_threshold=0.05,
        use_aabb=True, max_obstacle_width=0.0),
    GraspGenXModule.blueprint(gripper=C["gripper"],
        grasp_frame_to_tcp=C["grasp_frame_to_tcp"], max_candidates=20),
)
```

5. Blueprint / proposals, motion and agent

```

openyam_bench_proposals = autoconnect(
    *_sensing, OpenYamBenchMount.blueprint(),
).global_config(n_workers=4)

_model = make_openyam_model_config().model_copy(update={
    "base_pose": PoseStamped(frame_id="world"),
})
_hw = openyam_hardware()
openyam_bench_grasp = autoconnect(
    *_sensing,
    ManipulationModule.blueprint(model=_model, world_frame="world",
        static_transforms=[_mount], visualization={"backend": "viser"},
        default_speed_scale=0.1, linear_speed_scale=0.1),
    ManipulationSkills.blueprint(),
    PickAndPlaceModule.blueprint(planning_frame="world",
        max_grasp_attempts=1,
        grasp_verification={"empty_epsilon": C["empty_epsilon"]}),
    coordinator(hardware=[_hw], tasks=[
        trajectory_task(_hw),
        TaskConfig(name="openyam_gripper", type="gripper",
            joint_names=[OPENYAM_GRIPPER_JOINT], priority=20),
    ]),
).global_config(n_workers=6)

_PROMPT = """You control one six-joint OpenYAM and its gripper.
The D435i is fixed to the bench, not mounted on the wrist.
World is the robot base frame. Units: metres and radians.
Use get_robot_state for status. Use scan_objects with object names.
Use only exact object IDs returned by the latest successful scan.
Only call pick_object when the operator explicitly requests a pick.
Only call place_at after a successful pick and with operator-supplied
release TCP coordinates. Never invent target coordinates or handovers.
Never move home/init automatically. Never auto-reset or retry a fault.
After a failed tool call stop and report the error for inspection.
Do not release a held object except for an explicitly requested place.
A successful gripper check is not visual proof of a stable grasp.
When told to inspect or scan only, do not move the arm or gripper.
"""

openyam_bench_agent = autoconnect(
    openyam_bench_grasp,
    McpServer.blueprint(),
    McpClient.blueprint(system_prompt=_PROMPT,
        model=os.environ.get("OPENYAM_LLM_MODEL", McpClientConfig().model)),
).global_config(n_workers=6)

```

Before motion: the grasp and agent variants enable the real arm on startup. They do not yet contain your bench obstacles: add them on page 10 after every restart, before any movement command. The proposals variant has no coordinator and does not enable motors.

max_candidates=20 limits returned results **after inference**; it is not a GPU memory setting.
 max_grasp_attempts=1 prevents trying multiple candidates in a single first-run pick. The slow speed scales are starting values, not certified safety limits. The agent prompt is behavioral guidance, not a hard interlock.

6. Prove perception and grasps without motion

Register once, then start proposals only

```
export OPENYAM_BENCH_CONFIG="$PWD/openyam_bench.json"
python -m compileall -q dimos/robot/manipulators/openyam/blueprints/bench.py
pytest dimos/robot/test_all_blueprints_generation.py -q
git diff -- dimos/robot/all_blueprints.py
dimos run openyam-bench-proposals
```

The registration test can write the registry then fail because it is uncommitted. Proceed only if that is the sole failure and the diff adds the expected blueprints; fix actual import/schema errors. Commit your reviewed local files when ready. Do not set CI to suppress generation. [7]

First launch provisions the isolated GraspGenX Python runtime and downloads model assets. YOLOE text prompting also downloads its text encoder. Do this with internet and the arm off; keep the logs. [4, 10]

Second terminal: activate the same environment, then dimos shell

```
scene = app.ObjectSceneRegistrationModule
seen = scene.scan_scene(text=["apple"])
print(seen)
ids = [str(d.id) for d in seen.detections if d.id]
print(ids)
# Continue only if exactly one intended object is detected.
assert len(ids) == 1
cloud = scene.get_object_pointcloud_by_object_id(ids[0])
assert cloud is not None
points = cloud.points_f32()
print(cloud.frame_id, points.shape)
print(points.min(axis=0), points.max(axis=0))
proposals = app.GraspGenXModule.propose_grasps(cloud)
print(proposals.header.frame_id, len(proposals.candidates))
print([(c.score, c.pose) for c in proposals.candidates[:3]])
```

If the camera is still starting, wait for current RGB-D frames and rescan. Use a matte object that fits within the gripper aperture. Inspect its mask/cloud for tabletop contamination and finite coordinates in metres. A partial-view cloud centroid is not the object centre; compare known visible surfaces and bounds to your measured rig.

GPU acceptance test

```
# Third terminal: whole-device memory, including separate model processes.
watch -n 1 nvidia-smi
```

Repeat scans and proposals on representative objects. Pass only if predictions are finite, plausible and correctly framed, with no CUDA errors/OOM and adequate free memory under sustained use. Record latency and peak device usage; single-process Torch statistics miss the other process. No inference-rate target is promised. Stop this blueprint before starting the motion stack.

7. Enable the arm and perform one pick

Bolt down the arm, clear the workspace and have a tested physical stop/power control reachable. Support the arm when disabling torque. Do not treat unplugging CAN as an emergency stop. Run only one coordinator at a time. [2]

```
dimos hardware can list
dimos hardware can setup can0
ip -details -statistics link show can0
dimos run keyboard-teleop-openyam
# Stop teleop safely before the next run.
dimos run openyam-planner-coordinator
```

Confirm classical CAN at 1 Mbit/s, motor topology, correct joint direction, gripper endpoints and a small supervised movement. Verify the planned path before executing it; do not blindly send the original hard-coded home pose. Your CAN adapter must actually expose SocketCAN; do not assume its brand or firmware. Termination must match the supplied arm wiring.

Start the custom stack, then add the measured bench

```
# Stop the previous controller safely first.
dimos run openyam-bench-grasp
# In another activated terminal, from the project root:
export OPENYAM_BENCH_CONFIG="$PWD/openyam_bench.json"
dimos shell
```

In the shell, load the same JSON using its absolute path. Run this after every full-stack restart. Add similar boxes for mounts/clamps if they enter the workspace; then inspect the collision geometry in the viser viewer (default port 8095).

```
import json, os
from dimos.msgs.geometry_msgs.Pose import Pose
from dimos.msgs.geometry_msgs.Vector3 import Vector3
from dimos.msgs.geometry_msgs.Quaternion import Quaternion
cfg = json.load(open(os.environ["OPENYAM_BENCH_CONFIG"]))
bench = Pose(Vector3(*cfg["bench_center_m"]), Quaternion(0, 0, 0, 1))
print(app.ManipulationModule.add_obstacle(
    "bench", bench, "box", cfg["bench_size_m"]))
print(app.ManipulationModule.get_obstacles())
```

This assumes an axis-aligned bench. Model rotated fixtures with measured orientations. Do not substitute floor_z: upstream implements it as a small fixed-position box, not an infinite plane. Clear, modelled surroundings are required even though the final grasp leg remains unchecked. [5]

Scan, review, then pick - separate commands

```
scan = app.PickAndPlaceModule.scan_objects(["apple"])
print(scan)
# Inspect the result and select its exact ID; no motion yet.
# Replace OBJECT_ID only after reviewing the object and grasp path.
pick = app.PickAndPlaceModule.pick_object("OBJECT_ID")
print(pick)
# Only after success and visual confirmation of a stable hold:
# print(app.PickAndPlaceModule.place_at(*cfg["place_tcp_m"]))
```

Before pick_object, inspect grasp proposals as on page 9; show them with ManipulationModule.show_grasp_proposals(proposals). Check finger/table clearance along approach and retreat. place_at targets the TCP, keeps the selected grasp orientation, and does not account automatically for arbitrary held-object geometry. [5]

8. Add agentic control to the same stack

The agent orchestrates the tested skills; it does not replace grasp generation or motion planning. Enable it only after the manual pick/place sequence works. Choose a tool-calling model available to your API account and pass its exact identifier; the repository default is not a guarantee of account access. [6]

Start

```
# In the activated project shell; enter the API key without echoing it.
read -rs -p "OpenAI API key: " OPENAI_API_KEY
export OPENAI_API_KEY
# Optional: set OPENYAM_LLM_MODEL to your available model identifier.
export OPENYAM_BENCH_CONFIG="$PWD/openyam_bench.json"
# Stop the previous controller safely before starting this one.
dimos run openyam-bench-agent
```

In a second activated shell, re-add and inspect your measured bench/fixture obstacles using page 10. This is a new process; obstacles added to a previous run do not persist. Then inspect the MCP tools:

```
dimos mcp status
dimos mcp list-tools
dimos humancli
```

First prompts

1. "Report the robot state. Do not move the arm or gripper."
2. "Scan for an apple. Report the object IDs; do not pick anything."
3. After checking the scene: "Pick the apple from the latest scan. Stop after lifting; do not move home or release it."
4. After checking the hold: "Place it at [your measured release TCP x, y, z] in world metres. Stop afterward."

You can use **dimos agent-send "..."** for one-shot prompts, but humancli is easier for seeing responses. On a fault, stop and inspect before manually resetting; automatic reset/retry and automatic return-to-init were deliberately removed from this rig prompt.

What this first version does not do

It does not localize a basket and calculate a collision-safe placement inside it. It does not servo visually from the wrist camera, continuously track a moving target, or guarantee stable grasp verification from video. Gripper feedback checks whether the jaws stopped before closing fully; blockage and partial grasp can look similar.

If you mean completely offline

Keep the robotics modules as above, but a local tool-calling LLM requires a separate provider/backend configuration and a new memory/latency test. Do not assume it will coexist with YOLOE and GraspGenX in 8 GB VRAM. The easiest first agentic run uses a hosted LLM while all robotics inference remains on your Ubuntu machine.

Troubleshooting and source record

Symptom	Action
Prompt-free / set_classes error	Confirm prompt_mode=YoloePromptMode.PROMPT and non-pf weights.
GraspGenX OOM	Measure all processes. max_candidates only truncates output. If necessary, reduce upstream sampling in backend.py: pass num_grasps=64, min_grasps=10, max_tries=1 to run_inference; this is an optional source change requiring revalidation, not a guaranteed fix.
CUDA / unsupported kernel error	Retain the full isolated-runtime traceback; check its Torch/CUDA/driver compatibility. Host CUDA success alone is insufficient. Do not install FlashAttention as a default remedy.
Camera native build/access error	Build the camera-specific Rust flake, verify udev permissions and USB 3, and close competing camera processes.
Wrong pose / empty world cloud	Check world-to-camera_link, optical transforms, timestamp alignment and metre units. Do not move until resolved.
Missed detections	Inspect lighting, depth and mask. Default confidence is 0.6; lower carefully if needed. The detector also filters very large image boxes; do not place the object too close.
Pick falsely succeeds or fails	Calibrate normalized gripper feedback and thresholds; inspect mechanical grip. A stalled jaw does not prove a secure hold.

Later: add a self-filtered voxel map, persist fixture geometry in a module, enforce workspace/tool restrictions in code, validate grasp/contact recovery, then integrate the wrist view. A heuristic grasp provider can isolate planning faults if needed, but it does not satisfy the GraspGenX acceptance test.

Primary sources checked

- [1] Ubuntu installation and dependency selection
- [2] OpenYAM integration, topology and stopping behavior
- [3] RealSense native build and camera configuration
- [4] GraspGenX adapter and isolated runtime lock
- [5] Pick/place execution and collision-check behavior
- [6] MCP agent composition and gripper feedback
- [7] Blueprint registration behavior
- [8] NVIDIA CUDA 12.8 driver compatibility
- [9] GraspGenX pinned source; YAM status and sampler
- [10] Ultralytics YOLOE prompting and segmentation
- [11] dimOS object registration modes and masks
- [12] Gripper threshold calibration

Pinned model revision: adithyamurali/GraspGenXModel @ 7c834043c11a11417e31d6d5ea9355801e40a2c1. Source review and PDF QA complete; hardware acceptance remains to be performed on the actual rig.